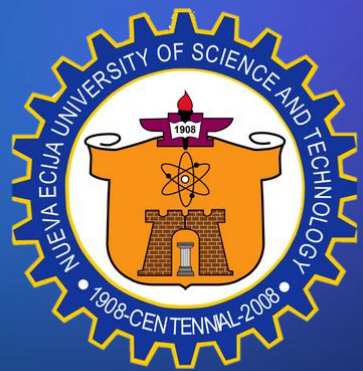




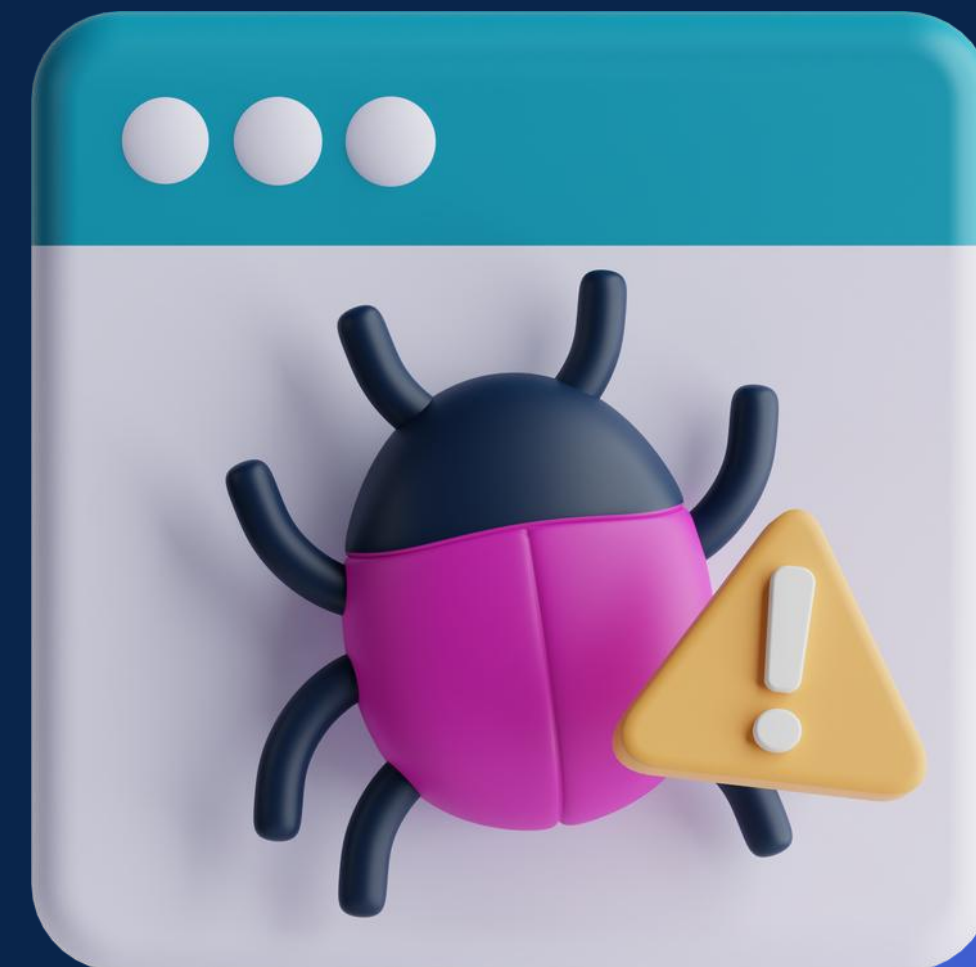
REPUBLIC OF THE PHILIPPINES
NUEVA ECIJA UNIVERSITY OF SCIENCE AND TECHNOLOGY
COLLEGE OF INFORMATION AND COMMUNICATIONS TECHNOLOGY



ITWS - 04

WEB SYSTEMS VULNERABILITIES

PREPARED BY:
JOHN PHILLIP M. MEDINA

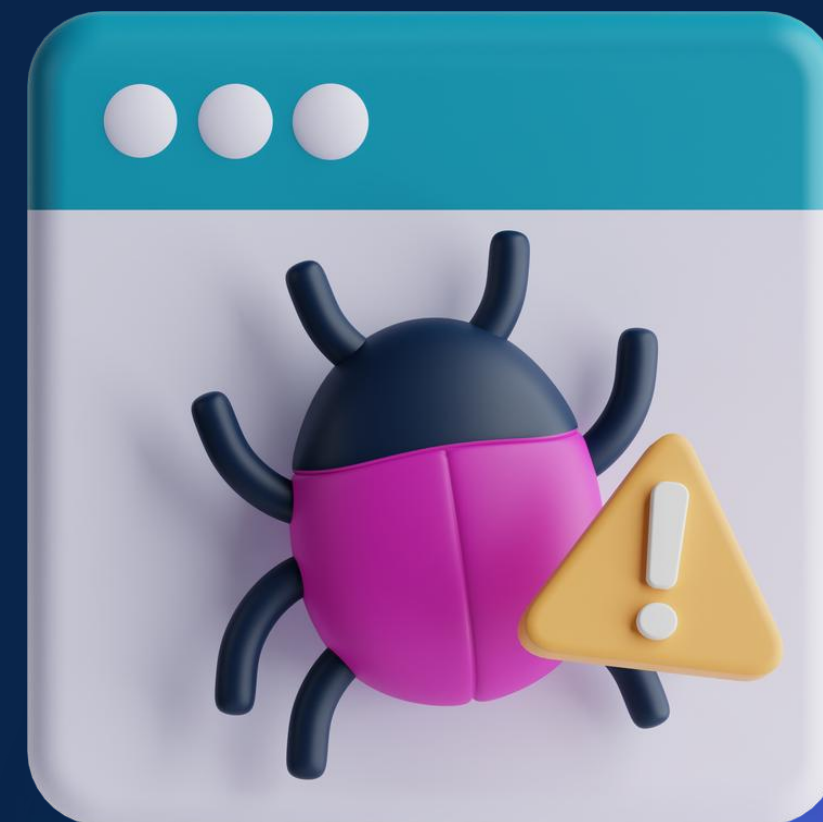




REPUBLIC OF THE PHILIPPINES
NUEVA ECIJA UNIVERSITY OF SCIENCE AND TECHNOLOGY
COLLEGE OF INFORMATION AND COMMUNICATIONS TECHNOLOGY

UNIT II

OPEN WEB APPLICATION SECURITY PROJECT (OWASP)

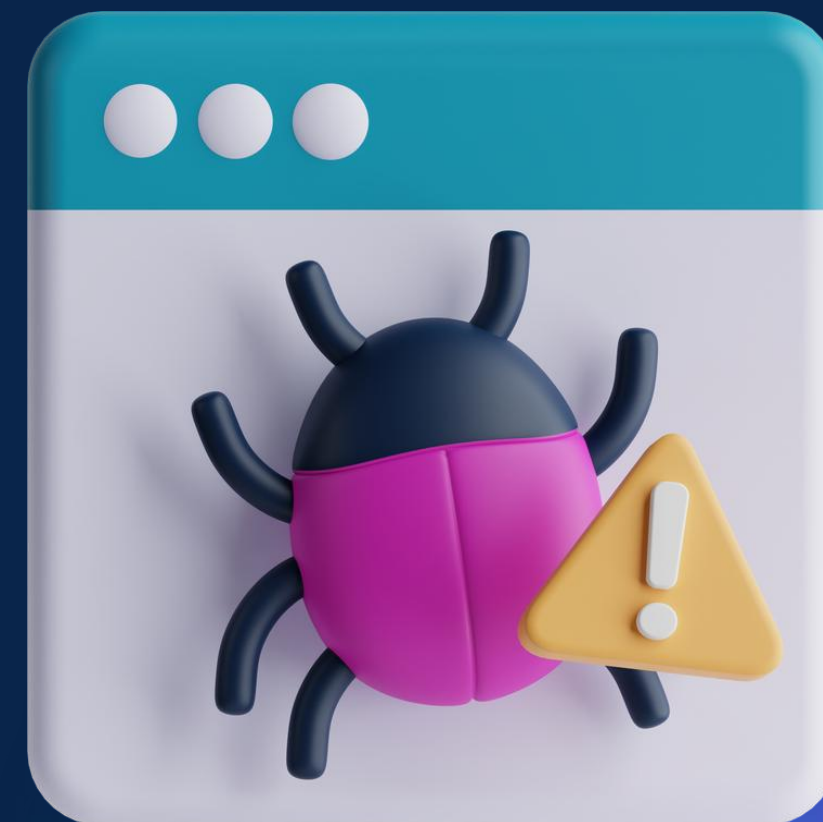




REPUBLIC OF THE PHILIPPINES
NUEVA ECIJA UNIVERSITY OF SCIENCE AND TECHNOLOGY
COLLEGE OF INFORMATION AND COMMUNICATIONS TECHNOLOGY

LESSON 4

OWASP TOP 10 A01:2025 - A05:2025



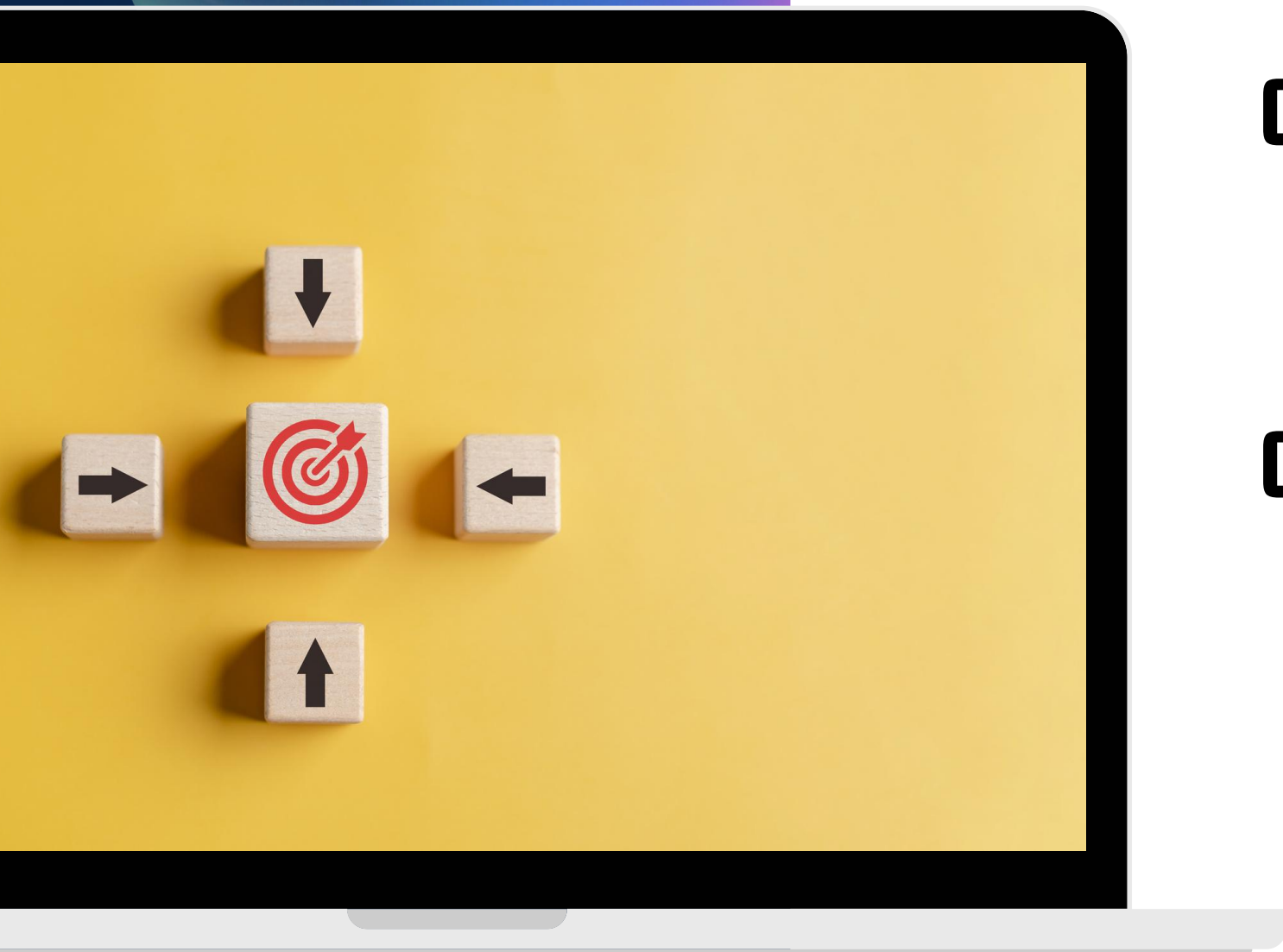
UNIT OBJECTIVES

01

Explain OWASP Top 10 vulnerabilities A01–A05 and their security impact on web applications.

02

Identify and analyze common attack scenarios related to access control, misconfiguration, cryptography, supply chain, and injection flaws.



OWASP Top 10 : 2025

1. [A01:2025 – Broken Access Control](#)
2. [A02:2025 – Security Misconfiguration](#)
3. [A03:2025 – Software Supply Chain Failures](#)
4. [A04:2025 – Cryptographic Failures](#)
5. [A05:2025 – Injection](#)
6. [A06:2025 – Insecure Design](#)
7. [A07:2025 – Authentication Failures](#)
8. [A08:2025 – Software or Data Integrity Failures](#)
9. [A09:2025 – Security Logging & Alerting Failures](#)
10. [A10:2025 – Mishandling of Exceptional Conditions](#)





A01:2025 **BROKEN ACCESS CONTROL**

Access control enforces policy such that users cannot act outside of their intended permissions.

Broken Access Control can lead to unauthorized access to sensitive data or functionality **due to improper implementation of role-based access controls (RBAC) or misconfigured permissions.**

A01:2025 **BROKEN ACCESS CONTROL**

Common access control vulnerabilities include:

- Access is granted too broadly instead of being limited to specific authorized users or roles.
- Attackers bypass security by modifying URLs, parameters, or request data.
- Users can access or modify another user's data by changing an identifier.
- APIs allow create, update, or delete actions without proper authorization checks.
- Users gain higher privileges or access accounts without proper authentication.

A01:2025 **BROKEN ACCESS CONTROL**

Example Attack Scenarios

Scenario 1: A school portal allows students to view their grades using a URL like this:

```
https://school.edu/portal/grades?studentId=20231045
```

The system **does not check** whether the logged-in student actually owns that studentId.

```
https://school.edu/portal/grades?studentId=20231001
```


A01:2025 **BROKEN ACCESS CONTROL**

Example Attack Scenarios

Scenario #2: An attacker simply forces browsers to target URLs. Admin rights are required for access to the admin page.

```
https://example.com/app/getappInfo  
https://example.com/app/admin_getappInfo
```

If an unauthenticated user can access either page, it's a flaw. If a non-admin can access the admin page, this is a flaw.



A02:2025 SECURITY MISCONFIGURATION

Security misconfiguration is when a system, application, or cloud service is set up incorrectly from a *security perspective*, creating vulnerabilities.

A02:2025 SECURITY MISCONFIGURATION

The application might be vulnerable if:

- Default accounts and their passwords are still enabled and unchanged.
- Unnecessary features are enabled or installed (e.g., unnecessary ports, services, pages, accounts, testing frameworks, or privileges).
- A lack of central configuration for intercepting excessive error messages. Error handling reveals stack traces or other overly informative error messages to users.

A02:2025 SECURITY MISCONFIGURATION

Example Attack Scenarios

Scenario #1

The server still has sample or default applications that should have been removed before deployment. These applications are known to be insecure. If default usernames and passwords are not changed, attackers can log in easily and gain full control of the system.

A02:2025 SECURITY MISCONFIGURATION

Example Attack Scenarios

Scenario #2

Directory listing is enabled, so attackers can see all files and folders on the server. They download compiled program files, decompile them, and study the source code. This allows attackers to discover hidden vulnerabilities such as missing access controls.

Index of /

<u>Name</u>	<u>Last modified</u>	<u>Size</u>	<u>Description</u>
 secret/	2017-01-27 15:40	-	
 priv/	2017-01-27 15:41	-	
 edit/	2017-01-27 15:40	-	
 dir1/	2017-01-27 15:40	-	
 config.php	2017-01-27 15:40	11K	

Apache/2.4.23 (Win64) PHP/5.6.25 Server at localhost Port 80



A03:2025 SOFTWARE SUPPLY CHAIN FAILURES

Refers to security risks that happen when an application **trusts third-party software, libraries, tools, or updates that have been tampered with, compromised, or poorly managed.**

In simple terms, the application becomes vulnerable **not because of its own code**, but because **something it depends on is unsafe.**

A03:2025 SOFTWARE SUPPLY CHAIN FAILURES

You are likely vulnerable if:

- Failing to track component versions means developers may unknowingly use outdated or vulnerable client-side, server-side, or transitive dependencies, increasing the risk of security breaches.
- Using vulnerable, unsupported, or outdated components
- Using components from untrusted sources

A03:2025 SOFTWARE SUPPLY CHAIN FAILURES

Example Attack Scenarios

Scenario 1: A trusted vendor is compromised with malware, leading to your computer systems being compromised when you upgrade. The most famous example of this is probably:

- The 2019 SolarWinds compromise that led to ~18,000 organizations being compromised. <https://www.npr.org/2021/04/16/985439655/a-worst-nightmare-cyberattack-the-untold-story-of-the-solarwinds-hack>

A03:2025 SOFTWARE SUPPLY CHAIN FAILURES

Example Attack Scenarios

Scenario 2: After a Samsung system update, some users noticed a permanent green vertical line on their phone screens. Users did not drop the phone, change hardware, or install third-party apps. The issue appeared only after installing the official update.

A03:2025 SOFTWARE SUPPLY CHAIN FAILURES

Example Attack Scenarios

Scenario 3: Cloudflare outage on November 18, 2025

These incidents, often referred to as "self-inflicted" outages, are a critical form of supply chain failure where a trusted vendor (Cloudflare) pushes a faulty update or configuration, breaking dependent systems.



A04:2025 CRYPTOGRAPHIC FAILURES

Cryptographic failures happen when sensitive data is *not properly protected* using **encryption**. This can be due to weak or outdated encryption methods, poor key management, or storing/transmitting data without encryption.

Attackers use **brute force, sniffing, or cryptanalysis** to break weak encryption or intercept sensitive data, leading to data breaches, financial fraud, and identity theft.

A04:2025 CRYPTOGRAPHIC FAILURES

Common cryptographic failure vulnerabilities include:

- Transmitting data in clear text.
- Old or weak cryptographic algorithms or protocols used either by default or in older source codes.
- Use of default crypto keys, weak crypto keys generated or re-used or missing proper key management or rotation.
- Use of deprecated hash functions such as MD5 or SHA1 when cryptographic hash functions are needed

A04:2025 CRYPTOGRAPHIC FAILURES

Example Attack Scenarios

Scenario #1: An application encrypts credit card numbers in a database using automatic database encryption. However, this data is automatically decrypted when retrieved, allowing a SQL injection flaw to retrieve credit card numbers in clear text

A04:2025 CRYPTOGRAPHIC FAILURES

Example Attack Scenarios

Scenario #2: A web-based banking site allows some pages to load over HTTP instead of HTTPS and supports weak encryption.

When a user connects using a public Wi-Fi network, an attacker:

- Listens to the network traffic
- Downgrades the connection from HTTPS to HTTP
- Steals the user's session cookie
- Reuses the cookie to take over the user's logged-in session

A04:2025 CRYPTOGRAPHIC FAILURES

Article

PDF Available

Evaluation of Common Security Vulnerabilities of State Universities and Colleges Websites Based on OWASP

April 2024 · [Journal of Electrical Systems](#) No. 20(No. 5s):1396-1404

DOI:[10.52783/jes.2471](#)

License · [CC BY-ND 4.0](#)

Authors:



Carlos Pineda Flores Jr.
Tarlac State University



Richard Naje Monreal
Mapúa University

In **2024**, a security study on **state universities and colleges' websites in the Philippines** found that many systems **did not implement proper encryption**, which **exposed sensitive information** and increased the risk of **data breaches and unauthorized access**.



A05:2025 INJECTION

An injection vulnerability is an application flaw that allows **untrusted user input** to be sent to an interpreter (e.g. a browser, database, the command line) and causes the **interpreter to execute parts of that input as commands**

A05:2025 INJECTION

An application is vulnerable to attack when:

- **No input validation or sanitization** – The system accepts user input as-is, even if it contains malicious code.
- **Unsafe dynamic queries** – The application builds database queries using user input instead of safe, parameterized queries.
- **Direct concatenation of hostile data** – User input is mixed with SQL or commands, letting attackers change how the query or command works

A05:2025 INJECTION

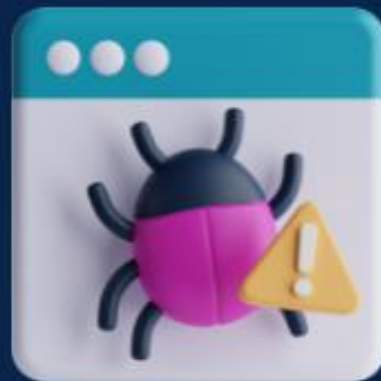
Example Attack Scenarios

Scenario #1: An application uses untrusted data in the construction of the following vulnerable SQL call:

```
String query = "SELECT * FROM accounts WHERE custID=" +  
request.getParameter("id") + "";
```

An attacker modifies the 'id' parameter value in their browser to send: ' OR '1'='1'.
For example:

```
http://example.com/app/accountView?id=' OR '1'='1'
```



ITWS - 04

WEB SYSTEMS VULNERABILITIES

END OF THE LECTURE

REFERENCES



IT-WS01:WEB SYSTEMS AND
TECHNOLOGIES 1 –MODULE 2026

PRESENTATION BY:
JOHN PHILLIP M. MEDINA
